Sollicitatie-voorbereiding: IaC, secrets en security monitoring

Oct 15, 2026 · @Tobias

Modelantwoorden op drie gespreksvragen, met de achtergrond om ze ook bij doorvragen te kunnen verdedigen.

Onbekend begrip? Alles staat uitgelegd in de Begrippenlijst; bij elke vraag staan links naar de bijbehorende thema's.

Vraag 1 — PostgreSQL in Kubernetes vanaf nul, volgens IaC

Begrippen bij deze vraag: IaC, GitOps en deployment · Kubernetes-bouwstenen · Secret-management-tools · Identiteit en toegang · Security-scanning

"Stel je hebt niets: geen repo, geen landing zone, geen cluster. Hoe zet je een PostgreSQL-instance in Kubernetes op vanuit IaC-principes?"

Kern van het antwoord: alles declaratief in git, in een vaste volgorde opgebouwd, en de database via een **operator** in plaats van zelfgebouwde StatefulSets.

De opbouw in zes stappen

1. **Fundament (bootstrap):** een git-repo en een remote Terraform-state, bijvoorbeeld een Azure Storage-account met versioning. Die state-opslag maak je met een kleine, aparte bootstrap-stap, omdat Terraform zijn eigen state niet kan aanmaken. CI via GitHub Actions met **OIDC-federatie** naar Azure: de pipeline krijgt kortlevende tokens, er staan geen credentials opgeslagen.
2. **Platform met Terraform:** netwerk, Key Vault en het cluster (AKS, of on-prem VM's met kubeadm).
3. **GitOps bootstrappen:** Terraform installeert Flux of Argo CD. Vanaf dat moment wordt alles in het cluster vanuit git uitgerold en bijgehouden; handmatige `kubectl apply` is niet meer nodig.
4. **Database via een operator: CloudNativePG** beheert de volledige lifecycle van een HA-PostgreSQL-cluster: streaming replication, automatische failover, continue backups naar object storage en point-in-time recovery. Sinds 2025 een CNCF Sandbox-project. Je beschrijft één `Cluster`-resource (aantal instances, StorageClass, backup-doel) en de operator doet de rest.
5. **Secrets:** DB-credentials staan in Key Vault en komen via External Secrets Operator in het cluster (zie vraag 2).

6. **Controle in CI:** Trivy scant Terraform-code en manifests op misconfiguraties en gelekte secrets vóór uitrol.

Waarom een operator en niet zelf een StatefulSet?

Onderwerp	Zelf (StatefulSet / simpele Helm-chart)	Operator (CloudNativePG)
Failover	Zelf bouwen of handmatig	Automatisch, primary/standby
Backups + restore	Zelf scripten (CronJob, pg_dump)	Continu naar object storage, PITR ingebouwd
Upgrades	Handmatig	Rolling updates
Gebruikers, databases, config	Handmatig of in scripts	Declaratief in de `Cluster`-resource

Waarom GitOps-sync naast CI/CD?

Push (CI/CD): de pipeline draait één keer na een merge, doet `kubectl apply` of `helm upgrade`, en is klaar. Wat er daarna in het cluster gebeurt, ziet hij niet meer. **Pull (GitOps-sync):** een agent **in** het cluster (Argo CD of Flux) vergelijkt continu git met wat er echt draait, en stuurt bij als het afwijkt.

Probleem	Alleen CI/CD (push)	Met sync (pull)
Drift (`kubectl edit`, `scale`, iets verdwijnt)	Onopgemerkt tot de volgende pipeline-run	Zichtbaar als *OutOfSync*; met `selfHeal` automatisch teruggezet
Verwijderen	Bestand uit git halen laat de resource in het cluster staan	Met `prune` verdwijnt de resource ook uit het cluster
Toegang en security	Pipeline heeft cluster-credentials nodig, en de runner moet de API-server kunnen bereiken	Agent haalt zelf op uit git: geen inkomende toegang, geen cluster-credentials in CI
Wat draait er echt?	Git = wat er *uitgerold zou zijn*	Git **is** de waarheid; rollback = `git revert`
Nieuw of verloren cluster	Alle pipelines opnieuw, in de juiste volgorde	Agent installeren, naar de repo wijzen, klaar

CI/CD verdwijnt niet, het krijgt een andere rol: CI bouwt, test en scant het image (Trivy) en zet de nieuwe image-tag in de manifest-repo. Argo CD of Flux synct die wijziging naar het cluster. *CI levert aan git, git levert aan het cluster.*

Eerlijk over de nadelen: een extra component om te draaien en te debuggen, en secrets moeten apart (ESO/SOPS). Voor één klein cluster is push prima; de waarde groeit met meer clusters, mensen en audit-eisen.

Vergelijking met PowerShell DSC

GitOps met Argo CD of Flux is in feite **DSC in pull-modus, voor Kubernetes**:

PowerShell DSC	Argo CD / Flux	Wat het doet
Configuratie (`.ps1` → MOF)	YAML-manifests, Helm-charts, Kustomize in git	Beschrijft de gewenste toestand
Pull-server / SMB-share	Git-repo (of OCI-registry)	Bron van de gewenste toestand
Local Configuration Manager (LCM) op de node	**Argo CD / Flux-controllers** in het cluster	Agent die ophaalt, vergelijkt en toepast
`RefreshMode = Push` (`Start-DscConfiguration`)	CI/CD met `kubectl apply`	Eenmalig van buitenaf toepassen
`RefreshMode = Pull`	GitOps-sync	Agent haalt zelf periodiek op
`ApplyOnly`	Push zonder sync	Eén keer toepassen, drift niet controleren
`ApplyAndMonitor` (standaard in DSC)	Sync **zonder** `selfHeal` (alleen *OutOfSync* tonen)	Drift rapporteren
`ApplyAndAutoCorrect`	Sync **met** `selfHeal`	Drift rapporteren én herstellen
`ConfigurationModeFrequencyMins` (standaard 15)	Sync-/reconcile-interval	Hoe vaak er gecontroleerd wordt

Verschillen: DSC configureert het **besturingssysteem** van een machine; Argo CD/Flux beheren **Kubernetes-objecten** via de API-server. Bij GitOps is git zelf de bron, dus je krijgt geschiedenis, pull requests en `git revert` erbij. En Kubernetes heeft zelf al een "LCM" ingebouwd (controllers houden bijv. replicas in stand); GitOps legt daar een laag bovenop: niet "3 Pods", maar "alles wat in git staat".

Zin voor in een gesprek: "GitOps is voor Kubernetes wat DSC in `ApplyAndAutoCorrect` is voor Windows-servers: de gewenste toestand staat centraal, een agent haalt die op en herstelt drift."

Bronnen: OpenGitOps-principes · Argo CD · Flux op AKS/Arc (pull, geen externe toegang) · DSC Local Configuration Manager

Wat staat er in de repo voor Postgres? (GitOps-kant)

De grens: Terraform bouwt alles **buiten** het cluster (netwerk, Key Vault, identities, het cluster zelf) en installeert Argo CD of Flux. Daarna komt alles **binnen** het cluster uit git: de operator, de secrets-koppeling, de database en de backups.

```
infra-repo/
├── terraform/                    # buiten het cluster (via GitHub Actions +
OIDC)
|   ├── network.tf  keyvault.tf  aks.tf  identity.tf
|   └── gitops-bootstrap.tf       # installeert Argo CD/Flux en wijst naar
gitops/
└── gitops/                       # binnen het cluster (Argo CD/Flux synct
dit)
    ├── root-app.yaml             # één Application die alle andere
Applications uitrolt
    ├── platform/
    |   ├── external-secrets.yaml  # wave -1: ESO-operator (Helm-chart)
    |   └── cloudnative-pg.yaml    # wave -1: CNPG-operator (Helm-chart)
    └── apps/shop-db/
        ├── secretstore.yaml       # wave 0: hoe ESO Key Vault bereikt
(Workload Identity)
        ├── externalsecret.yaml    # wave 1: DB-credentials uit Key Vault ->
Kubernetes Secret
        ├── cluster.yaml           # wave 2: de PostgreSQL-Cluster (3
instances)
        └── scheduledbackup.yaml   # wave 3: dagelijkse backup
```

Volgorde is essentieel. Een `Cluster`-resource kan pas bestaan als de CNPG-operator (en zijn CRD) er is, en de database heeft het Secret nodig voordat hij start. Dat regel je zo:

- **Argo CD:** de annotatie `argocd.argoproj.io/sync-wave`. Lagere waves eerst (standaard 0, negatief mag), en Argo CD gaat pas door als alles synced en healthy is.
- **Flux:** `dependsOn` in een `Kustomization`. Die wordt pas toegepast als de afhankelijke Kustomizations `Ready` zijn.

Voorbeeld: de operator als Argo CD-Application (wave -1) en de database zelf (wave 2):

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: cloudnative-pg
  namespace: argocd
  annotations: {argocd.argoproj.io/sync-wave: "-1"}
spec:
  project: default
  source:
    repoURL: https://cloudnative-pg.github.io/charts
    chart: cloudnative-pg
    targetRevision: <vaste chartversie>      # pinnen, nooit 'latest'
  destination: {server: https://kubernetes.default.svc, namespace: cnpg-
system}
  syncPolicy:
    automated: {prune: true, selfHeal: true}
    syncOptions: [CreateNamespace=true, ServerSideApply=true]
---
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: shop-db
  annotations: {argocd.argoproj.io/sync-wave: "2"}
spec:
  instances: 3
  storage: {size: 10Gi}
  # credentials en backup: zie hieronder
```

Hetzelfde met **Flux**. Flux kent geen waves; je knipt de repo in mappen en maakt per map een `Kustomization` (een Flux-CR, niet te verwarren met het `kustomization.yaml`-bestand van Kustomize). Met `dependsOn` leg je de volgorde vast, met `wait: true` wacht Flux tot alles in die map healthy is:

```
# clusters/prod/platform.yaml  -> operators eerst (ESO, CNPG)
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata: {name: platform, namespace: flux-system}
spec:
  interval: 10m
  path: ./gitops/platform
  prune: true
  wait: true                        # pas Ready als de operators draaien
  sourceRef: {kind: GitRepository, name: flux-system}
---
# clusters/prod/shop-db.yaml  -> database pas na de operators
```

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata: {name: shop-db, namespace: flux-system}
spec:
  dependsOn: [{name: platform}]    # vergelijkbaar met een hogere sync-wave
  interval: 10m
  path: ./gitops/apps/shop-db
  prune: true                      # ~ Argo CD 'prune'
  wait: true
  sourceRef: {kind: GitRepository, name: flux-system}
```

De CNPG-operator zelf staat in `gitops/platform/` als Helm-chart, via twee Flux-CR's:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata: {name: cnpg, namespace: cnpg-system}
spec:
  interval: 1h
  url: https://cloudnative-pg.github.io/charts
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata: {name: cloudnative-pg, namespace: cnpg-system}
spec:
  interval: 10m
  chart:
    spec:
      chart: cloudnative-pg
      version: <vaste chartversie>   # pinnen, nooit 'latest'
      sourceRef: {kind: HelmRepository, name: cnpg}
```

De `Cluster`-YAML in `gitops/apps/shop-db/` is identiek aan die bij Argo CD, alleen zonder de sync-wave-annotatie. Selfheal hoef je bij Flux niet aan te zetten: elke `interval` past Flux de staat uit git opnieuw toe.

	Argo CD	Flux
Wat wordt gesynct	`Application` (CR)	`Kustomization` of `HelmRelease` (CR's)
Helm-chart	`source.chart` in de Application	`HelmRepository` + `HelmRelease`
Volgorde	`sync-wave`-annotatie per resource	`dependsOn` tussen Kustomizations

	Argo CD	Flux
Wachten op healthy	standaard per wave	`wait: true` of `healthChecks`
Drift terugzetten	`selfHeal: true`	elke `interval`

Bronnen: Flux Kustomization · Flux HelmRelease

Wat dit oplevert in de praktijk:

- `instances: 3` → `5` in een PR, na review mergen → Argo CD synct → de operator schaalt. Geen handwerk, en de geschiedenis staat in git.
- Iemand verandert met de hand iets aan de database-resources → Argo CD (met `selfHeal`) zet het terug.
- Nieuw cluster nodig (bijv. disaster recovery)? Terraform draaien → Argo CD synct `gitops/` → operator, secrets en database komen terug. Met een `bootstrap` vanuit de backup krijg je ook de data terug.

Twee details die laten zien dat je bijblijft:

- **Backups:** CloudNativePG ondersteunt nog de ingebouwde backup naar object storage, maar die is **sinds versie 1.26 deprecated** ten gunste van de **Barman Cloud Plugin**. Plan nieuwe opzetten dus met de plugin, en een `ScheduledBackup`-resource voor het schema.
- **Credentials:** het wachtwoord staat **niet** in de repo. `externalsecret.yaml` verwijst alleen naar de naam van het secret in Key Vault; ESO maakt het Kubernetes Secret dat de Cluster gebruikt.

Bronnen: Argo CD sync waves · Flux Kustomization (dependsOn) · CloudNativePG-chart · CloudNativePG backups (deprecatie, plugin)

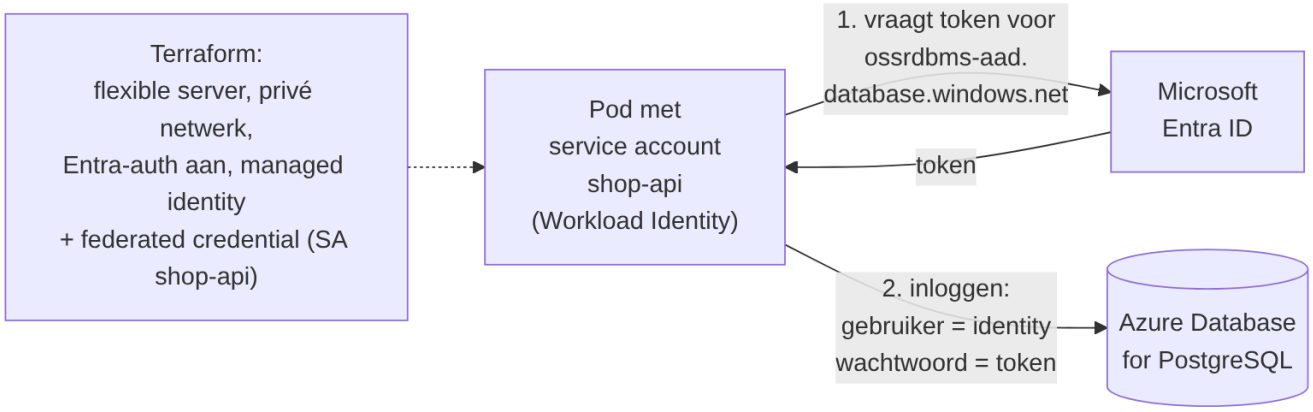
Pluspunt om te noemen

In Azure zou je ook kritisch afwegen of een **managed database** (Azure Database for PostgreSQL) niet beter past dan Postgres in Kubernetes: minder beheer, maar minder controle. Dat laat zien dat je niet alleen het *hoe* kent, maar ook het *of*. *(Eigen afweging, geen bronfeit.)*

Alternatief: managed Postgres in Azure, gebruikt vanuit Kubernetes

Begrippen: Kubernetes-bouwstenen (Azure Database for PostgreSQL, ASO, Service account) · Identiteit en toegang (Managed identity, Workload identity federation) · IaC, GitOps en deployment (Terraform)

Hier draait de database **niet** in het cluster. Azure Database for PostgreSQL flexible server beheert Microsoft; Kubernetes draait alleen de app. CloudNativePG heb je dan niet nodig.



Wie doet wat:

1. **Terraform** maakt de server in een privé netwerk, zet Microsoft Entra-authenticatie aan en wijst een Entra-admin aan. Het maakt ook een user-assigned managed identity met een federated credential voor het service account van de app (hetzelfde mechanisme als ESO bij Key Vault).
2. **Eenmalig in de database** (als Entra-admin, op de `postgres`-database) maak je een rol voor die identity:

```
select * from pgaadauth_create_principal('<identity_name>', false, false);
```

3. **De app in Kubernetes** draait met dat service account. Via Workload Identity vraagt hij een Entra-token aan voor `https://ossrdbms-aad.database.windows.net/.default` en gebruikt dat token als wachtwoord. In code doet de Azure SDK (`DefaultAzureCredential`) dit voor je.
4. **Het resultaat:** er bestaat nergens een databasewachtwoord. Je hoeft dus niets te bewaren, niets te roteren en het kan niet lekken. ESO is voor de database niet nodig.

Kan de app geen Entra-tokens gebruiken? Dan gebruik je wachtwoordauthenticatie. Terraform zet het wachtwoord in Key Vault, liefst met `ephemeral` of een write-only attribuut zodat het niet in de state komt. ESO maakt er een Kubernetes Secret van, precies zoals bij vraag 2. Een `ExternalName`-Service kan de servernaam in het cluster een vaste interne naam geven (`shop-db`), zodat de app niet weet dat de database buiten het cluster staat.

Alles vanuit git, ook de server zelf? Dat kan met Azure Service Operator (ASO): een operator met CRD's voor Azure-resources (meer dan 150 soorten, waaronder PostgreSQL). Dan maakt Argo CD of Flux de server via een CR in plaats van Terraform. Meestal laat je Terraform de Azure-kant doen en GitOps de clusterkant; ASO is de optie als het team alles in Kubernetes-YAML wil.

	CloudNativePG in het cluster	Azure Database for PostgreSQL
Wie beheert Postgres	jij (via de operator)	Microsoft
Waar staan de data	PV's in het cluster	buiten het cluster, in Azure
Inloggen vanuit de app	wachtwoord (Secret van CNPG of ESO)	Entra-token via Workload Identity, zonder wachtwoord
Backups	Barman Cloud Plugin + `ScheduledBackup`	door Azure
Controle	volledig (versie, extensies, tuning)	beperkt tot wat Azure aanbiedt
Past bij	on-prem, hybride (bijv. Azure Local), veel controle	Azure-native, weinig beheer

In het gesprek: "In Azure kies ik liever een flexible server met Entra-auth: de app logt via Workload Identity in met een token, dus er bestaat geen wachtwoord. CloudNativePG kies ik als de database on-prem of op Azure Local moet draaien of als we volledige controle nodig hebben."

Bronnen: Verbinden met managed identity (Microsoft Learn) · Azure Service Operator

Modelantwoord (± 1 minuut)

"Ik begin met een repo en een remote Terraform-state, en CI met OIDC naar Azure zodat er geen secrets in de pipeline staan. Terraform bouwt alles buiten het cluster: netwerk, Key Vault, identities en het cluster zelf, en installeert Argo CD of Flux. Vanaf dan komt alles binnen het cluster uit git: in sync-waves eerst de CloudNativePG- en External Secrets-operators, dan de koppeling met Key Vault, dan de PostgreSQL-Cluster-resource met replicas, en als laatste de backup-planning via de Barman Cloud Plugin. Wijzigingen gaan via een pull request, Argo CD synct ze en herstelt drift. Trivy scant alles in CI. En ik zou eerst afwegen of een managed Postgres-dienst niet beter past."

Vraag 2 — Secret management (naast Docker/Kubernetes Secrets)

Begrippen bij deze vraag: Kubernetes-bouwstenen · Secret-management-tools · Identiteit en toegang · Security-scanning · IaC, GitOps en deployment

Kern van het antwoord: denk in vier lagen. Dan kun je elke tool plaatsen en uitleggen waarom je hem kiest.

Laag	Vraag	Tools
Bron van waarheid	Waar staat het secret echt?	Azure Key Vault, HashiCorp Vault
Levering aan Kubernetes	Hoe komt het in de Pod?	External Secrets Operator, Secrets Store CSI Driver, Sealed Secrets, SOPS
Bescherming in het cluster	Wie kan het lezen?	RBAC, encryption at rest voor etcd
Detectie van lekken	Staat er ergens een secret waar het niet hoort?	Trivy, GitHub secret scanning

Kubernetes Secrets: wat ze wel en niet zijn

Een Secret is standaard alleen **base64-gecodeerd**, niet versleuteld, en zo opgeslagen in etcd, tenzij je *encryption at rest* instelt. Wie het object mag lezen, kan de waarde dus direct terugzien:

```
kubectl create secret generic db-pw --from-literal=password=Geheim123
kubectl get secret db-pw --output=jsonpath='{.data.password}' | base64 --
decode   # -> Geheim123
```

- De echte beveiliging zit in **RBAC**: wie mag `get` op secrets in welke namespace.
- Gebruik in een Pod: als omgevingsvariabele (`env.valueFrom.secretKeyRef`) of als gemount bestand (`volumes.secret`). Een gemount bestand wordt bijgewerkt als het Secret verandert; een env-var pas na een herstart.
- Daarom wil je Secrets **niet** met de hand aanmaken en **niet** onversleuteld in git zetten. Daar zijn de tools hieronder voor.

De vier leveringstools

Tool	Hoe het werkt	Secret in git?	Wanneer kiezen
External Secrets Operator (ESO)	Leest uit Key Vault (via `SecretStore` / `ClusterSecretStore`) en maakt via een `ExternalSecret` een gewoon Kubernetes Secret; haalt periodiek opnieuw op (`refreshInterval`)	Nee, alleen de verwijzing	Key Vault als enige bron van waarheid, GitOps-vriendelijk

Tool	Hoe het werkt	Secret in git?	Wanneer kiezen
Secrets Store CSI Driver (Azure-provider)	Mount secrets uit Key Vault direct als bestanden in de Pod; optioneel sync naar een Secret; automatische rotatie mogelijk	Nee	Secret hoeft niet als Kubernetes-object te bestaan
Sealed Secrets	`kubeseal` versleutelt met de publieke sleutel van de controller; alleen de controller in het cluster kan ontsleutelen	Ja, versleuteld (`SealedSecret`)	Geen externe vault, alles in git
SOPS	Versleutelt waarden in YAML/JSON-bestanden met Azure Key Vault, age, PGP of cloud-KMS; CNCF Sandbox	Ja, versleuteld	Versleutelde config in git, ook buiten Kubernetes

Aandachtspunten bij doorvragen:

- **ESO-authenticatie:** aanbevolen is **Workload Identity** (geen opgeslagen wachtwoord). Minimale rol: *Key Vault Secrets User*.
- **Sealed Secrets:** zonder backup van de privésleutel van de controller ben je je secrets kwijt. De sleutel wordt standaard elke 30 dagen vernieuwd, maar dat is géén rotatie van je eigenlijke wachtwoorden.

Combineren met wat je al kent

1. **Key Vault** is de bron van waarheid.
2. **GitHub Actions** authenticeert via **OIDC** naar Azure: geen opgeslagen secrets in GitHub nodig voor de deploy zelf.
3. **ESO** (met Workload Identity) levert de secrets aan het cluster.
4. **Trivy** in CI en **GitHub secret scanning** bewaken dat er niets in de repo lekt.

Van `${{ secrets.X }}` naar External Secrets: de verbanden

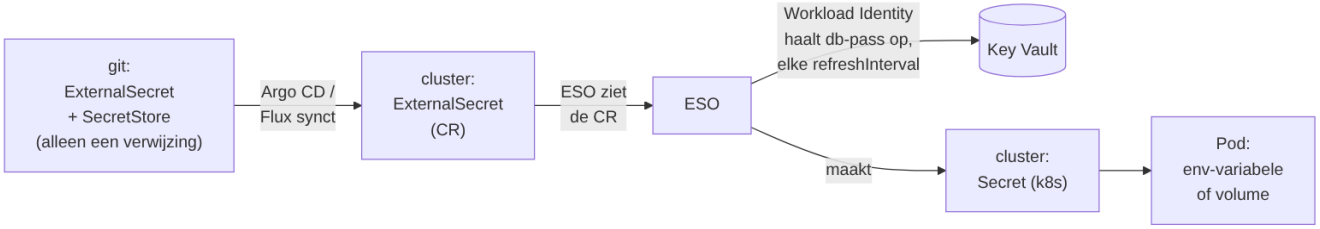
Je vertrekpunt: een waarde staat ergens opgeslagen (`.env`, `.tfvars`, GitHub-secret) en wordt bij het draaien als **omgevingsvariabele** aan een proces gegeven. Dat blijft ook bij de geavanceerde technieken het eindpunt. Wat verandert: **waar de waarde woont**, **wie hem ophaalt** en **wanneer**.

Niveau	Waar staat de waarde?	Wie haalt hem op, wanneer?	Hoe komt hij in het proces?	Aandachtspunt
1. Lokaal bestand	`.env` / `.tfvars` op je laptop (gitignored)	Jij, bij het starten	env-var of `-var-file`	Lekt makkelijk via git of je laptop
2. GitHub Actions secrets	In GitHub	De runner, bij de start van de job	`env: DB_PW: ${{ secrets.DB_PW }}`	Kopie per repo; vaak ook langlevende cloud-credentials als secret
3. OIDC + Key Vault in de pipeline	Key Vault	De job logt via **OIDC** in bij Azure (geen opgeslagen credentials) en leest het secret tijdens de run	env-var in de job, of een Terraform-data source	Terraform zet gelezen waarden in de **state** (plaintext), tenzij je `ephemeral` (1.10+) of write-only-argumenten (1.11+) gebruikt
4. External Secrets Operator	Key Vault	De ESO-controller in het cluster, **periodiek** (`refreshInterval`), met Workload Identity	ESO schrijft een Kubernetes Secret → de Pod leest het als env-var (`secretKeyRef`) of bestand	De waarde gaat nooit door je pipeline of git: daar staat alleen de **naam** van het secret
5. Secrets Store CSI Driver	Key Vault	De driver, bij het **starten van de Pod** (plus rotatie)	Als bestand in de Pod	Geen Kubernetes Secret nodig
6. App haalt zelf op	Key Vault	De applicatie zelf, tijdens runtime, met een managed identity	In het geheugen van de app	Vereist code in de app

Architectuur: wie haalt het secret uit de vault?

Begrippen: Kubernetes-bouwstenen (CR, CRD, controller) · Secret-management-tools (ESO, SecretStore, ExternalSecret, Sealed Secrets, SOPS) · IaC, GitOps en deployment (Argo CD, Flux) · Identiteit en toegang (Workload identity federation)

Een veelgemaakte denkfout: "Argo CD of Flux synct het secret rechtstreeks uit de vault." Dat klopt niet. **Argo CD en Flux komen nooit bij de vault.** Ze syncen alleen een *verwijzing*; External Secrets Operator (ESO) haalt de waarde op.



Wie doet wat:

1. **In git** staan een `SecretStore` (welke vault, welke identity) en een `ExternalSecret` (welk secret, welke Kubernetes Secret). Er staat geen waarde in.
2. **Argo CD of Flux** zet die twee CR's in het cluster, net als een Deployment. Daarmee is het werk van Argo of Flux klaar.
3. **ESO** ziet de `ExternalSecret`, logt via Workload Identity in bij Entra, leest de waarde uit Key Vault en maakt er een gewone Kubernetes `Secret` van.
4. **ESO blijft pollen** volgens `refreshInterval` (bijv. `1h`; met `0` haalt hij de waarde maar één keer op). Roteer je het secret in Key Vault, dan werkt ESO de Kubernetes Secret vanzelf bij, zonder commit of sync.
5. **De Pod** gebruikt de Secret zoals altijd. Een waarde als env-variabele verandert pas na een herstart van de Pod; een gemounte volume wordt na een tijdje vanzelf bijgewerkt.

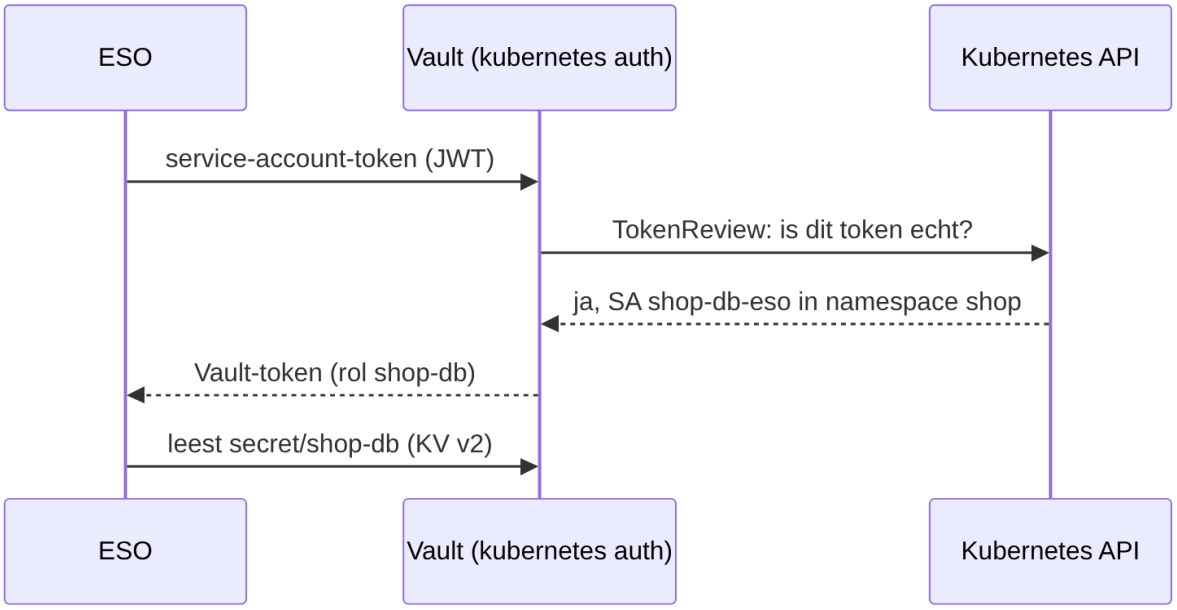
Twee aparte loops: GitOps synct de *configuratie* (welk secret, waar vandaan), ESO synct de *waarde*. Daardoor staat er nooit een secret in git, in de Redis-cache van Argo CD of in Terraform state.

Wat CR betekent: een Custom Resource is een object van een type dat niet standaard in Kubernetes zit, maar met een CRD is toegevoegd. De ESO-installatie levert de CRD's (`ExternalSecret`, `SecretStore`), jouw YAML in git is de CR, en de ESO-controller voert hem uit. Hetzelfde patroon zie je bij Argo CD (`Application`), Flux (`Kustomization`), CloudNativePG (`Cluster`), MetalLB (`IPAddressPool`) en Gateway API (`Gateway`, `HTTPRoute`). Op het cluster: `kubectl get crd` en `kubectl explain <type>.spec`.

Variant: HashiCorp Vault in plaats van Key Vault

Begrippen: Secret-management-tools (HashiCorp Vault, SecretStore, ExternalSecret) · Kubernetes-bouwstenen (Service account, RBAC)

De architectuur blijft gelijk: git bevat alleen de verwijzing, ESO haalt de waarde op. Alleen de `SecretStore` verandert, want ESO moet nu bij Vault inloggen in plaats van bij Entra.



SecretStore met Kubernetes-auth (zo logt ESO in met de identity van een service account, zonder statisch token):

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata: {name: vault-backend, namespace: shop}
spec:
  provider:
    vault:
      server: https://vault.example.org
      path: secret          # mount van de KV-engine
      version: v2           # KV v2 (standaard)
      auth:
        kubernetes:
          mountPath: kubernetes   # waar de kubernetes-auth in Vault staat
          role: shop-db           # Vault-rol: welke SA mag welke policy
          serviceAccountRef: {name: shop-db-eso}
```

ExternalSecret (gelijk aan de Key Vault-versie, alleen een andere store):

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata: {name: shop-db-credentials, namespace: shop}
spec:
  refreshInterval: 1h
  secretStoreRef: {name: vault-backend, kind: SecretStore}
  target: {name: shop-db-credentials}  # Secret die ESO maakt
  data:
  - secretKey: password                # sleutel in die Secret
    remoteRef:                         # = secret/shop-db, veld password
      key: shop-db
      property: password
```

	Azure Key Vault	HashiCorp Vault
Hoe ESO inlogt	Workload Identity (federated OIDC-token naar Entra)	Kubernetes-auth (SA-token, Vault controleert via TokenReview); ook token, AppRole, JWT/OIDC, LDAP, cert, AWS IAM
Rechten geregeld in	Azure RBAC op de vault	Vault-policy gekoppeld aan een Vault-rol
Wat ESO kan lezen	secrets, keys, certificaten	alleen de KV-engine (v1 en v2); andere engines via de Vault Generator
Extra in het cluster	niets	de SA die Vault gebruikt voor TokenReview heeft de ClusterRole `system:auth-delegator` nodig
Terugschrijven (PushSecret)	ja	ja, met create/read/update op het pad

Twee valkuilen die laten zien dat je de details kent:

- **Audience:** sinds Vault 1.21 moet een Kubernetes-auth-rol een audience hebben (1.20 waarschuwt alleen). Bij een upgrade breekt het inloggen anders.
- **Statische tokens vermijden:** `tokenSecretRef` werkt, maar dan staat er een langlevend Vault-token als Secret in het cluster. Kubernetes-auth of JWT/OIDC gebruikt kortlevende tokens; dat is hetzelfde principe als Workload Identity bij Azure.

Bron: [ESO: HashiCorp Vault provider]

Syncen Argo CD en Flux ook secrets?

Ja, maar nooit een gewoon `Secret`-manifest in git: base64 is geen versleuteling. Wat in git staat, bepaalt hoe veilig het is.

In git staat	Wie maakt de echte Secret?	Argo CD	Flux
`ExternalSecret`: alleen een verwijzing (ESO)	ESO, met de waarde uit Key Vault via Workload Identity	✅ aanbevolen	✅
`SealedSecret`: versleuteld met de publieke key van de controller	Sealed Secrets-controller in het cluster	✅ aanbevolen	✅
SOPS-versleuteld `Secret`	Flux: kustomize-controller ontsleutelt bij toepassen (`spec.decryption.provider: sops`; age, PGP, Vault of Azure Key Vault)	⚠️ alleen via een plugin	✅ ingebouwd

Argo CD *"strongly recommend[s]"* secrets in het doelcluster te laten maken (ESO, Sealed Secrets, CSI-driver) en waarschuwt tegen invullen tijdens het genereren van manifests (zoals argocd-vault-plugin): die manifests worden in plaintext gecachet in Redis, en een secret-wijziging gaat pas mee met een app-sync.

In het gesprek: "Argo of Flux synct alleen een verwijzing of iets versleuteld. De echte waarde blijft in Key Vault; ESO maakt er in het cluster een Secret van en houdt hem bij."

Bronnen: Argo CD: Secret Management · Flux: SOPS · ESO: ExternalSecret · Kubernetes: Custom Resources

Waar staan secrets, en hoe zijn ze versleuteld?

Begrippen: Kubernetes-bouwstenen (Encryption at rest, Envelope encryption, tmpfs) · Secret-management-tools (Sealed Secrets, SOPS, Secrets Store CSI Driver)

Een secret komt onderweg op een aantal plekken langs. Per plek: wat staat er, en is het versleuteld?

Plek	Wat staat er	Versleuteld?	Hoe actueel
Git	ESO: alleen `ExternalSecret` / `SecretStore`, geen waarde. Sealed Secrets/SOPS: versleutelde waarde	ESO: niets te versleutelen. Sealed/SOPS: ja	bij elke commit
Key Vault	de echte waarde	ja, door Azure; HSM-keys bij Premium. Transport TLS 1.2/1.3; afschermen met private endpoint of firewall	bron van waarheid
`SecretStore` (CR in etcd)	alleen de verbinding: welke vault, welke identity. Bij Workload Identity geen credential	n.v.t.	via GitOps
ESO-controller	haalt de waarde via TLS op en schrijft de Kubernetes Secret	in transit (TLS)	elke `refreshInterval`
Kubernetes Secret (etcd)	de waarde, base64	**standaard niet**; base64 is geen versleuteling	door ESO bijgewerkt
Node (kubelet)	alleen op nodes met een Pod die de Secret gebruikt; volumes in `tmpfs` (RAM, niet op schijf)	nee, maar alleen in geheugen	standaard Watch: volume na een vertraging bijgewerkt
Pod	env-variabele of bestand	nee	env: pas na herstart; volume: vanzelf; `subPath`: nooit

De zwakke schakel is etcd. Of je nu ESO, Sealed Secrets of SOPS gebruikt, het eindresultaat is altijd een gewone Kubernetes Secret in etcd. Kubernetes zegt het zelf: *"Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store (etcd)."* Encryption at rest regel je zo:

- **kubeadm (je lab):** een `EncryptionConfiguration`-bestand en de vlag `--encryption-provider-config` op de kube-apiserver. Van de providers is `kms` v2 het sterkst (stable sinds 1.29). `aescbc` geldt als zwak, `aesgcm` alleen met automatische key-rotatie, en `identity` versleutelt niets. Bestaande secrets versleutel je door ze te herschrijven: `kubectl get secrets --all-namespaces -o json | kubectl replace -f -`.
- **AKS:** Azure versleutelt de opslag altijd (AES-256). KMS-etcd-encryptie voegt daar een laag aan toe, maar is opt-in: secrets worden versleuteld voordat ze in etcd komen, met *envelope encryption*. Een DEK versleutelt de secret, en een KEK in Key Vault versleutelt de DEK. Let op: verwijder je de key of de vault, dan zijn de secrets onherstelbaar.

Zie het zelf in je lab (op cp1). Dit toont het wachtwoord leesbaar uit etcd zolang encryption at rest uit staat:

```
kubectl create secret generic test --from-literal=pw=geheim123 && \
sudo etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/test | strings | grep geheim123
```

Versleutelen en ontsleutelen per tool:

Tool	Versleuteld door	Ontsleuteld door	Daarna
ESO	niet nodig: in git staat geen waarde	n.v.t.; Key Vault levert via TLS	Kubernetes Secret in etcd
Sealed Secrets	jij, met `kubeseal` en de **publieke** key van de controller (asymmetrisch)	de controller in het cluster, met zijn **private** key. Keys vernieuwen elke 30 dagen; strict scope: naam en namespace moeten kloppen	Kubernetes Secret in etcd
SOPS + Flux	jij, met `sops` en een age-, PGP- of Azure Key Vault-key	de kustomize-controller van Flux, bij het toepassen	Kubernetes Secret in etcd

Tool	Versleuteld door	Ontsleuteld door	Daarna
Secrets Store CSI Driver	n.v.t.	Key Vault levert via TLS bij het starten van de Pod	standaard **geen** Kubernetes Secret: alleen een bestand in de Pod (tenzij `secretObjects`). Rotatie op AKS standaard elke 2 minuten

In het gesprek: "De waarde leeft in Key Vault. ESO maakt er een Kubernetes Secret van; die staat standaard onversleuteld in etcd, dus ik zet encryption at rest aan (op AKS KMS met Key Vault), beperk RBAC op secrets en mount ze liever als volume dan als env-variabele, zodat rotatie zonder herstart doorkomt."

Bronnen: Kubernetes Secrets · Encrypting data at rest · AKS KMS-concepten · AKS KMS etcd-encryptie · CSI-driver op AKS · Key Vault-beveiliging · Sealed Secrets · ESO-overzicht

Je vragen direct beantwoord

"Sla je met de operator dat geheel over?" Nee. Het eindpunt is hetzelfde: een env-var met een waarde in de container. ESO vervangt alleen de stap *"wie zet die waarde klaar"*. In plaats van dat jij of je pipeline de waarde ergens heen kopieert, haalt het cluster hem zelf uit Key Vault.

"Gaat hij met een token naar een secure store, op het moment van de run?" Bijna. Twee nuances:

- **Ja, met een token, en zonder opgeslagen wachtwoord.** ESO krijgt een kortlevend service-account-token van het cluster en ruilt dat bij Microsoft Entra in voor een toegangstoken. Dat is **precies hetzelfde mechanisme** als jouw GitHub Actions-OIDC naar Azure: *workload identity federation*. Entra vertrouwt de token-uitgever (`issuer`), de identiteit (`subject`) en het `audience`. Bij GitHub is de uitgever GitHub; bij ESO is het de OIDC-issuer van je cluster en het subject het service account.
- **Niet per run, maar vooraf en periodiek.** ESO synchroniseert op de achtergrond (bijv. elk uur) naar een Kubernetes Secret. De Pod leest dat Secret bij het starten. Wil je dat het secret pas bij het starten van de Pod wordt opgehaald, dan is de CSI-driver (niveau 5) dichter bij "op het moment van draaien".

Zo ziet het er IaC-first uit

1. **Terraform** (gedraaid door GitHub Actions via OIDC) maakt: de Key Vault, een managed identity, een *federated credential* die het service account van ESO vertrouwt, en de rol *Key Vault Secrets User*. Het secret zelf zet je in Key Vault zonder het in git te zetten. Let

op de state: gebruik `ephemeral`/write-only waar dat kan.

2. **Git (GitOps)** bevat alleen verwijzingen, geen waarden:

```
# ExternalSecret: "maak Secret 'db-credentials' uit Key Vault-secret 'db-
password'"
# (controleer de apiVersion bij jouw ESO-versie)
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    kind: SecretStore
    name: azure-keyvault
  target:
    name: db-credentials
  data:
    - secretKey: password
      remoteRef:
        key: db-password
```

3. **ESO** haalt de waarde op en maakt het Kubernetes Secret `db-credentials`.
4. **De Pod** gebruikt het als env-var, precies zoals je gewend bent:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: db-credentials
        key: password
```

Vergelijk met je GitHub-workflow: `env: DB_PASSWORD: ${{ secrets.DB_PASSWORD }}`. Zelfde idee, andere bron.

Modelantwoord (± 45 s): "In mijn pipelines gebruik ik OIDC naar Azure, zodat er geen langlevende credentials in GitHub staan, en Key Vault als bron van waarheid. Voor Kubernetes haal ik secrets niet via de pipeline naar binnen, maar laat ik het cluster ze zelf ophalen met External Secrets Operator. Die gebruikt Workload Identity, hetzelfde federatiemechanisme als GitHub-OIDC, en synchroniseert naar een Kubernetes Secret dat de Pod als env-var leest. In git staan dus alleen verwijzingen. En in Terraform let ik erop dat secrets niet in de state belanden."

Trivy in het kort

Trivy scant **container-images, filesystems, git-repo's, VM-images en Kubernetes-clusters**, en vindt **kwetsbaarheden (CVE's), IaC-misconfiguraties, secrets en licenties**; het kan ook een **SBOM** maken.

```
trivy image nginx:1.27                    # kwetsbaarheden in een image
trivy fs --scanners secret,misconfig .    # secrets + misconfiguraties in je
repo/Terraform
```

Waar draait Trivy? (van IDE tot cluster)

Trivy is een **security-scanner**, geen formatter of linter. `terraform fmt` of Prettier controleren opmaak; Trivy zoekt kwetsbaarheden, misconfiguraties en gelekte secrets. Je zet hem op meerdere plekken in, van vroeg ("shift left") tot runtime:

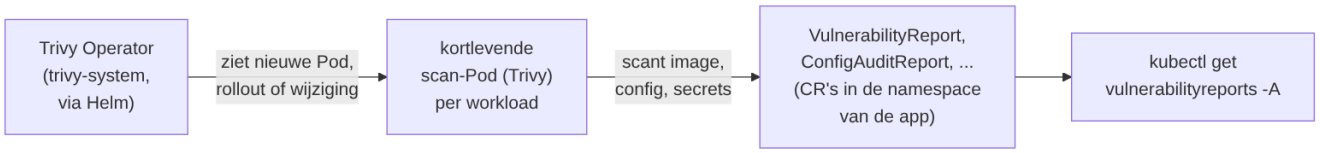
Plek	Hoe	Doel	Blokkeert het?
IDE	VS Code-plugin (onderdeel van het Trivy-ecosysteem)	Directe feedback tijdens het schrijven van Dockerfiles en Terraform	Nee, alleen meldingen
Lokaal / pre-commit	`trivy fs --scanners secret,misconfig --exit-code 1 .` in een git pre-commit-hook	Een secret onderscheppen vóórdat het ooit in git staat	Ja: de commit faalt bij een vondst
CI (pull request)	GitHub Action `trivy-action`: `scan-type` `fs`, `config` (IaC) of `image`; `exit-code: 1` laat de job falen; uitvoer als SARIF naar GitHub code scanning	Elke PR automatisch scannen; vondsten zichtbaar in het Security-tabblad van de repo	Ja, als je `exit-code: 1` zet (standaard `0` = alleen rapporteren)
Na de image-build	`trivy image` op het net gebouwde image, vóór de push naar het register	Geen kwetsbare images uitrollen	Ja, als stap in de pipeline
In het cluster (runtime)	**Trivy Operator**: scant continu en automatisch (bijv. bij een nieuwe Pod); resultaten als Kubernetes-resources (CRD's)	Vulnerabilities, config-audits, blootgestelde secrets, RBAC, compliance (CIS, NSA/CISA, Pod Security Standards)	Nee, het rapporteert; blokkeren doe je met admission-policies

Waarom op meerdere plekken? Elke laag vangt iets anders: de IDE en pre-commit voorkomen dat een secret in git komt, CI dwingt het af voor iedereen, en de operator vangt nieuwe CVE's in images die gisteren nog schoon waren. En een secret dat ooit gecommit is, moet je altijd **roteren**: verwijderen uit git is niet genoeg.

Trivy Operator: hoe werkt het in het cluster?

Begrippen: Security-scanning (Trivy, Trivy Operator, CVE, SBOM) · Kubernetes-bouwstenen (Operator, CR, CRD)

Moet je zelf een Pod opstarten? Nee. Je installeert de operator één keer, en die start zelf de scan-Pods. Het is hetzelfde patroon als ESO en CNPG: een controller let op het cluster en doet het werk.



Zo werkt het:

1. **Installeren:** met Helm in de namespace `trivy-system`. Daarna begint de operator meteen met scannen.
2. **Triggers:** de operator kijkt naar wijzigingen in het cluster. Er komt een nieuwe Pod of een rollout, en dan start hij een scan. Hij vindt alle images die in het cluster draaien, ook die van systeem-Pods.
3. **Resultaat als CR's:** de bevindingen komen als CR's in etcd, naast de workload: `VulnerabilityReport` (CVE's in images), `ConfigAuditReport` (misconfiguraties), `ExposedSecretReport` (secrets in images), `RbacAssessmentReport`, `InfraAssessmentReport`, `ClusterComplianceReport` (bijv. CIS) en `SbomReport`.
4. **Levenscyclus:** een report hoort bij de controller van de workload (bijv. de ReplicaSet). Komt er een nieuwe revisie, dan volgt een nieuwe scan; verdwijnt de oude revisie, dan ruimt garbage collection het oude report op.

In je lab proberen (op cp1, na `./addons.sh helm`):

```
helm install trivy-operator oci://ghcr.io/aquasecurity/helm-charts/trivy-operator --namespace trivy-system --create-namespace --version 0.32.1
```

```
kubectl get vulnerabilityreports,configauditreports,exposedsecretreports --all-namespaces --output=wide
```

Wil je toch één keer handmatig scannen, zonder operator? Dan kun je wel een losse Pod draaien met de Trivy-image. Die is na afloop weg en maakt geen reports:

```
kubectl run trivy --rm --stdin --tty --restart=Never --image=aquasec/trivy --image nginx:1.27
```

Valkuilen in productie:

- **etcd vol:** reports staan in etcd, en elke update is een nieuwe revisie. Een report met honderden CVE's kan groter worden dan 1,5 MiB, de limiet van de API-server. Gebruik een report-TTL (`OPERATOR_SCANNER_REPORT_TTL`), filter op severity (`trivy.severity: HIGH,CRITICAL`) of zet reports op een Persistent Volume (dan kun je ze niet meer met `kubectl` opvragen). Doe ook etcd compaction en defrag.
- **Standalone vs Client-Server:** standaard downloadt elke scan-Pod de volledige vulnerability-database. Met een gedeelde Trivy-server (Client-Server) bespaar je netwerkverkeer en voorkom je rate-limiting.

In het gesprek: "Trivy in de pipeline vangt problemen vóór de deploy. De Trivy Operator vangt nieuwe CVE's in images die al draaien, en zet ze als CR's in het cluster. Die kan ik met Prometheus of Sentinel uitlezen. Ik beperk wel de report-grootte, anders loopt etcd vol."

Bronnen: Trivy Operator (Aqua) · Vulnerability scanning · Aqua-blog: Trivy en etcd-uitputting

Modelantwoord (± 1 minuut)

"Ik zie vier lagen. Key Vault is de bron van waarheid. Naar Kubernetes lever ik met External Secrets Operator via Workload Identity, of met de CSI-driver als het secret alleen als bestand nodig is. Wil je alles in git zonder externe vault, dan Sealed Secrets of SOPS. In het cluster beperk ik toegang met RBAC en zet ik encryption at rest aan, want Kubernetes Secrets zijn standaard alleen base64. En met Trivy en GitHub secret scanning voorkom ik dat secrets in code of images belanden."

Vraag 3 — Een gelekte Key Vault-key monitoren

Begrippen bij deze vraag: Monitoring en incidentrespons · Identiteit en toegang · Certificaten (PKI)

"Hoe zou je een security-incident monitoren waarbij keys uit een Key Vault zijn gelekt?"

Kern van het antwoord: Azure Monitor/Log Analytics is de **datalaag**; **Microsoft Sentinel** is de **detectie- en responslaag** daarbovenop. Je antwoord (Azure Monitor en logging) was dus niet fout, alleen onvolledig.

Azure Monitor vs Sentinel

	Azure Monitor / Log Analytics	Microsoft Sentinel
Wat is het	Platform voor logs en metrics, met alerts	SIEM + SOAR, draait op een Log Analytics-workspace
Doel	Verzamelen, bevragen (KQL), waarschuwen	Signalen uit veel bronnen correleren tot **incidents**, onderzoeken, automatisch reageren
Detectie	Losse alertregels per signaal	Analytics-regels, threat intelligence, MITRE ATT&CK-dekking
Onderzoek	Queries	Entity graph, hunting-queries, notebooks

	Azure Monitor / Log Analytics	Microsoft Sentinel
Respons	Action groups (mail, webhook)	Automation rules en **playbooks** (Logic Apps), bijv. ticket aanmaken of account blokkeren

De keten voor een gelekte key

1. **Loggen:** Key Vault logt in de categorie `AuditEvent`: per operatie (bijv. `SecretGet`) de tijd, het resultaat, het **IP-adres** en de **identiteit** van de aanroeper. Met een *diagnostic setting* stuur je dat naar de Log Analytics-workspace waar Sentinel op draait.
2. **Detecteren: Defender for Key Vault** signaleert ongebruikelijke toegang en noemt de bron (object-ID, UPN of IP). In Sentinel correleer je dat met bijvoorbeeld Entra-sign-in-logs. Een typische detectieregel: ongewoon veel secret-opvragingen door één identiteit in korte tijd.
3. **Onderzoeken:** welke secrets zijn opgevraagd, wanneer, door wie, en wat deed die identiteit daarvoor (eerdere operaties in de auditlogs).
4. **Reageren:** een playbook schakelt het secret uit of roteert het, ontneemt de identiteit de toegang (of zet de Key Vault-firewall strakker), en maakt een ticket aan. Daarna laten applicatie-eigenaren nagaan waar de gelekte credentials zijn gebruikt.

Stappen 3 en 4 volgen de responsaanpak die Microsoft zelf adviseert bij Defender for Key Vault-alerts: bron vaststellen, toegang blokkeren, impact meten, roteren.

Goed om te weten

- Sentinel verhuist naar de **Microsoft Defender-portal**. Na **31 maart 2027** wordt het in de Azure-portal niet meer ondersteund.
- Een gelekte key komt vaak niet uit de vault zelf, maar uit code of een pipeline. Daarom hoort ook **preventie** in je antwoord: GitHub secret scanning, Trivy en kortlevende credentials via OIDC/Workload Identity.

Verdieping: een gelekte certificaatketen

Eerst de nuance: de keten zelf (leaf → intermediate → root) is **publiek**; die stuurt elke server bij elke TLS-verbinding mee. Wat gevaarlijk is, is een gelekte **private key**. Hoe erg dat is, hangt af van wélke key lekt: hoe hoger in de keten, hoe groter de schade.

Wat lekt	Gevolg	Detectie	Respons
Private key van een leaf-certificaat (bijv. TLS-cert in Key Vault)	Aanvaller kan zich voordoen als die ene dienst	Key Vault-auditlogs: de private key haal je op via het **secret**-endpoint met dezelfde naam als het certificaat (alleen als de key exporteerbaar is). Een Sentinel-regel op ongebruikelijke identiteiten of IP's die dat secret opvragen	Certificaat intrekken bij de CA (revocation), opnieuw uitgeven met een **nieuwe** key, toegang van de lekbron intrekken
Certificaat-credential van een service principal (app-registratie die met een cert inlogt)	Aanvaller logt in als die applicatie, met al haar rechten	**Entra ID Protection** voor workload identities: *Leaked Credentials* (bijv. gevonden in een publieke GitHub-repo) en *Suspicious Sign-ins* (ongewoon IP, land, resource of credential-type). Sign-in-logs van service principals naar Sentinel	Nieuwe credential toevoegen, de gelekte verwijderen; bij twijfel álle credentials vervangen en de Key Vault-secrets roteren waar die service principal bij kon
Private key van een intermediate of root CA (eigen PKI of de Kubernetes-cluster-CA)	Alles wat eronder ondertekend is, is niet meer te vertrouwen; aanvaller kan zelf geldige certificaten maken	Publieke CA's: **Certificate Transparency-monitors** melden nieuwe certificaten voor jouw domeinen die je niet zelf hebt aangevraagd. Eigen PKI: uitgifte-logs van de CA bewaken	Intermediate intrekken en alle onderliggende certificaten opnieuw uitgeven. **Kubernetes** ondersteunt geen revocation van client-certificaten: bij een gelekte cluster-CA moet je de hele CA roteren, een ingrijpende operatie

Hoe Sentinel/Defender hier passen: hetzelfde patroon als bij de gelekte key. *Ingest* (Key Vault-AuditEvent, Entra-sign-in-logs van service principals, Defender- en ID Protection-alerts) → *analytics-regels* → *incident* → *playbook* (credential van de app intrekken, secret uitschakelen, ticket). Een CT-monitor-feed naar Sentinel vraagt een eigen koppeling *(eigen inschatting)*.

Preventie, het sterkste deel van je antwoord:

- Maak private keys **niet-exporteerbaar**; HSM-keys zijn dat altijd. Dan kán de key niet uit de vault lekken.
- Liever **kortlevende credentials** (Workload Identity/OIDC) dan langlevende certificaten of secrets.
- In Kubernetes: gebruik voor gebruikers **OIDC** in plaats van client-certificaten, juist omdat die niet in te trekken zijn.

Modelantwoord (± 45 s): "Bij een certificaatketen kijk ik eerst wélke private key gelekt is. Een leaf-key: intrekken en opnieuw uitgeven; detectie via Key Vault-auditlogs, want de private key haal je via het secret-endpoint op. Een app-certificaat: Entra ID Protection meldt gelekte credentials en verdachte sign-ins, en dat correleer ik in Sentinel; respons is de credential vervangen. Een CA-key is het ergst: alles eronder opnieuw uitgeven, en in Kubernetes zelfs de hele CA roteren, omdat client-certificaten niet in te trekken zijn. Daarom: niet-exporteerbare keys en liever kortlevende credentials."

Exporteerbaarheid in Key Vault: wat kan eruit?

Of een private key Key Vault kan verlaten, hangt af van het soort object en van de certificate policy.

Object	Kan de private key eruit?
Certificate, keytype RSA/EC, policy `exportable: true`	**Ja.** De volledige PFX/PEM met private key is op te halen via het secret met dezelfde naam (`az keyvault secret download`), **zonder wachtwoord**: Key Vault bewaart het importwachtwoord niet
Certificate, policy `exportable: false` of keytype RSA-HSM/EC-HSM	Nee. Het secret bevat dan alleen het publieke deel
Key (`az keyvault key`)	Normaal niet. Uitzondering: Secure Key Release (Premium of Managed HSM, `--exportable true` plus een release-policy); de key gaat dan alleen naar een attested TEE, niet naar een beheerder

Object	Kan de private key eruit?
Secret	Altijd leesbaar voor wie Get-rechten heeft

Per certificaat maak je de key niet-exporteerbaar met `exportable: false` in de certificate policy, of met een HSM-keytype. Controleer wat je huidige standaard is:

```
az keyvault certificate get-default-policy | jq '.keyProperties'
```

Afdwingen over alle vaults: er is geen ingebouwde Azure Policy die "exportable moet uit" controleert. Wel kan: de ingebouwde policy *Certificates should use allowed key types* met alleen RSA-HSM en EC-HSM toegestaan en effect Deny. HSM-keys zijn altijd niet-exporteerbaar, dus zo dwing je het indirect af. Daar is een Premium-vault voor nodig.

Na het exporteren: of de key in de Windows cert store exporteerbaar is, bepaal je bij het importeren, niet in Key Vault. Een PFX zegt daar zelf niets over.

Import via	Standaard	Niet-exporteerbaar maken
`Import-PfxCertificate`	Niet exporteerbaar (alleen met `-Exportable` wel)	Niets doen
`certutil -importPFX`	Waarschijnlijk exporteerbaar (niet expliciet gedocumenteerd)	`certutil -importPFX bestand.pfx NoExport`
Import-wizard	Vinkje "Mark this key as exportable"	Vinkje uit laten

Niet-exporteerbaar in Windows is gemak, geen beveiligingsgrens: een lokale admin krijgt de key er met tools als mimikatz toch uit. Beter is handmatig exporteren helemaal te vermijden. De Key Vault VM-extensie (v4) zet certificaten zelf in de cert store, vernieuwt ze automatisch en slaat private keys altijd niet-exporteerbaar op (`keyExportable` is in v4 verwijderd).

In het gesprek: "Is een certificaat in Key Vault exporteerbaar, dan krijgt iedereen met leesrechten op het secret de PFX zonder wachtwoord. Ik zet `exportable: false` of HSM-keytypes in de policy, dwing HSM-keytypes af met Azure Policy, en rol certificaten uit met de VM-extensie in plaats van handmatig."

Bronnen: Key Vault-certificaten · Certificaat exporteren · Secure Key Release · Import-PfxCertificate · certutil NoExport · Key Vault VM-extensie

Modelantwoord (± 1 minuut)

"Azure Monitor en Log Analytics zijn de basis: ik zet de AuditEvent-logs van Key Vault via een diagnostic setting naar een Log Analytics-workspace. Daarbovenop gebruik ik Microsoft Sentinel als SIEM: Defender for Key Vault-alerts en analytics-regels, bijvoorbeeld op ongewoon veel secret-opvragingen, worden gecorreleerd met Entra-sign-ins tot één incident. Een playbook roteert of blokkeert het secret, trekt de toegang van de identiteit in en maakt een ticket. En ik voorkom lekken met secret scanning en kortlevende credentials."

Spiekbriefje

Vraag	Kernzin	Termen die je moet laten vallen
Postgres vanaf nul	"Declaratief in git, Terraform voor het platform, GitOps voor het cluster, een operator voor de database."	remote state, OIDC, Flux/Argo CD, CloudNativePG, PITR, managed alternatief
Secret management	"Vier lagen: bron, levering, bescherming, detectie."	Key Vault, External Secrets Operator, Workload Identity, CSI driver, Sealed Secrets, SOPS, RBAC, encryption at rest, Trivy
Gelekte key	"Monitor verzamelt, Sentinel detecteert, correleert en reageert."	AuditEvent, diagnostic setting, Defender for Key Vault, analytics rule, incident, playbook, rotatie

Begrippenlijst

Alle begrippen uit dit document, per thema. Per begrip: wat het is, waar mogelijk vergeleken met iets wat je al kent, en een link om verder te lezen.

IaC, GitOps en deployment

Begrip	Uitleg	Lees meer
Infrastructure as Code (IaC)	Infrastructuur beschrijven in code (bestanden in git) in plaats van met de hand klikken. Herhaalbaar, reviewbaar, versiebeheer	Terraform intro

Begrip	Uitleg	Lees meer
Terraform	IaC-tool van HashiCorp: je beschrijft resources (bijv. een Key Vault) en Terraform maakt, wijzigt of verwijdert ze	Terraform
Terraform state	Het administratiebestand waarin Terraform bijhoudt welke echte resource bij welke regel code hoort. Kan secrets bevatten	State
Remote state / backend	State niet lokaal maar centraal opslaan (bijv. Azure Storage), met locking, zodat een team samen kan werken	State
GitOps	Werkwijze: de gewenste toestand staat declaratief in git, en een agent in het cluster haalt die op en houdt het cluster er continu gelijk aan. Vier principes: declaratief, geversioneerd, automatisch gepulld, continu bijgestuurd	OpenGitOps
Flux	GitOps-tool voor Kubernetes: controllers in het cluster halen config uit git (of Helm/OCI) en passen die toe. CNCF Graduated	Flux
Argo CD	GitOps-tool voor Kubernetes met een web-UI: vergelijkt de live toestand met git, toont verschillen ("OutOfSync") en synchroniseert. Onderdeel van het CNCF Graduated Argo-project	Argo CD · CNCF
Application (Argo CD) / sync-wave	Argo CD-CR die zegt: deze bron (git-map of Helm-chart) hoort in dit cluster en deze namespace. De annotatie argocd.argoproj.io/sync-wave bepaalt de volgorde: lagere waves eerst	Sync waves
Helm	Pakketbeheerder voor Kubernetes. Een **chart** is een pakket (sjablonen + standaardwaarden); installeren maakt een **release**; charts staan in een **repository**. Vergelijk: `apt` voor Kubernetes-apps	Helm

Begrip	Uitleg	Lees meer
Kustomize	YAML aanpassen zonder sjablonen: een basis plus "overlays" per omgeving (bijv. prod = meer replicas). Ingebouwd in `kubectl apply --kustomize`	Kustomize
Kustomization (Flux) / dependsOn	Flux-CR die een map uit git periodiek (interval) toepast op het cluster. Niet hetzelfde als het kustomization.yaml-bestand van Kustomize. Met dependsOn wacht hij tot andere Kustomizations Ready zijn; het Flux-alternatief voor sync waves	Flux Kustomization
HelmRepository / HelmRelease (Flux)	Flux-CR's voor Helm: HelmRepository zegt waar de charts staan, HelmRelease welke chart in welke versie geïnstalleerd moet worden. Flux voert dan zelf helm install/upgrade uit	Flux HelmRelease
CI/CD	Continuous Integration (code automatisch testen/scannen bij elke wijziging) en Continuous Delivery (automatisch uitrollen)	GitHub Actions
Pre-commit-hook	Script dat git uitvoert vóór elke commit; faalt het, dan gaat de commit niet door	git hooks
Shift left	Controles zo vroeg mogelijk doen (in de IDE of bij de commit) in plaats van pas na de uitrol	—

Kubernetes-bouwstenen

Begrip	Uitleg	Lees meer
Operator	Een controller met domeinkennis die een complexe applicatie beheert (installeren, failover, backups, upgrades), aangestuurd via eigen resources	Operator pattern

Begrip	Uitleg	Lees meer
CRD (CustomResourceDefinition)	Een nieuw resourcetype toevoegen aan de Kubernetes-API, bijv. `Cluster` van CloudNativePG of `ExternalSecret`	CRD's
CR (Custom Resource)	Een object van een type dat een CRD heeft toegevoegd, bijv. jouw ExternalSecret-YAML. Kubernetes slaat het op in etcd maar doet er zelf niets mee; dat doet de controller. Vergelijk: CRD = tabeldefinitie, CR = rij, controller = programma dat de tabel uitleest	Custom Resources
Controller	Een loop die de gewenste staat (spec van een object) vergelijkt met de echte staat en het verschil wegwerkt. Ingebouwd (bijv. Deployment-controller) of meegeleverd met een CRD (bijv. ESO). Een operator is een controller met domeinkennis	Controllers
CloudNativePG	Operator die PostgreSQL in Kubernetes beheert: replicatie, automatische failover, backups naar object storage, PITR. CNCF Sandbox sinds 2025	CloudNativePG
Azure Database for PostgreSQL (flexible server)	Managed PostgreSQL van Microsoft, buiten het cluster. Met Entra-authenticatie logt een app in met een token in plaats van een wachtwoord	Managed identity-login

Begrip	Uitleg	Lees meer
Azure Service Operator (ASO)	Operator die Azure-resources (meer dan 150 soorten) beheert vanuit Kubernetes-CR's. Alternatief voor Terraform als je ook de Azure-kant via GitOps wilt doen	ASO
StatefulSet	Workload voor apps met vaste identiteit: vaste Pod-namen (`db-0`) en per Pod een eigen volume	StatefulSets
Streaming replication / failover	Een standby-database ontvangt continu wijzigingen van de primary; valt de primary uit, dan wordt de standby automatisch primary	CloudNativePG
PITR (point-in-time recovery)	Een database terugzetten naar een exact tijdstip, op basis van een backup plus de doorlopende transactielogs	CloudNativePG
Object storage	Opslag van bestanden als objecten via een API (bijv. Azure Blob, S3); vaak gebruikt als backup-doel	—
StorageClass	Beschrijft een soort opslag waaruit Kubernetes automatisch volumes kan aanmaken	Storage Classes
Kubernetes Secret	Object voor gevoelige waarden; standaard alleen base64-gecodeerd, niet versleuteld	Secrets
base64	Een codering (geen versleuteling): iedereen kan het direct terugvertalen met `base64 --decode`	—

Begrip	Uitleg	Lees meer
RBAC	Role-Based Access Control: wie (gebruiker/service account) mag wat (werkwoord) op welke resource	RBAC
Encryption at rest	Secrets versleuteld opslaan in etcd, zodat een kopie van de database ze niet prijsgeeft	Encrypting data
Envelope encryption (DEK/KEK)	Twee sleutels: een DEK (data encryption key) versleutelt de data, een KEK (key encryption key) in een KMS zoals Key Vault versleutelt de DEK. De KEK verlaat de KMS nooit. Zo werkt KMS v2 in Kubernetes en KMS-etcd-encryptie op AKS	AKS KMS-concepten
tmpfs	Bestandssysteem in het RAM. De kubelet zet Secret-volumes op tmpfs, zodat de waarde niet op de schijf van de node komt	Kubernetes Secrets
Service account	Identiteit voor processen in een Pod (zoals een gebruikersaccount, maar dan voor software)	Service Accounts
Admission policy	Regels die de API-server toepast vóórdat een object wordt opgeslagen, bijv. "geen containers als root"	Admission control

Secret-management-tools

Begrip	Uitleg	Lees meer
Azure Key Vault	Azure-dienst voor het veilig opslaan van secrets, sleutels en certificaten, met toegang via Entra en RBAC	Key Vault

Begrip	Uitleg	Lees meer
HashiCorp Vault	Zelf te hosten secret store, cloud-onafhankelijk; kan ook kortlevende (dynamische) credentials uitgeven	Vault
External Secrets Operator (ESO)	Operator die secrets uit een externe store (bijv. Key Vault) leest en er Kubernetes Secrets van maakt. `SecretStore` = hoe verbinden; `ExternalSecret` = welk secret ophalen	ESO + Key Vault
SecretStore / ClusterSecretStore	ESO-CR die de verbinding met de vault beschrijft: welke vault en met welke identity. SecretStore geldt binnen één namespace, ClusterSecretStore voor het hele cluster	ESO API
ExternalSecret	ESO-CR die zegt welk secret uit de vault in welke Kubernetes Secret moet komen, en hoe vaak opnieuw (refreshInterval). Bevat geen waarde, dus veilig in git	ESO API
Secrets Store CSI Driver	Mount secrets uit Key Vault als bestanden in een Pod, bij het starten van de Pod	CSI-driver op AKS
Sealed Secrets	Versleutelde secrets die veilig in git mogen; alleen de controller in het cluster kan ze ontsleutelen (met `kubeseal` versleutel je)	Sealed Secrets
SOPS	Versleutelt waarden in YAML/JSON-bestanden in git, met Key Vault, age, PGP of cloud-KMS als sleutel. CNCF Sandbox	SOPS
age	Eenvoudige, moderne tool voor bestandsversleuteling; vaak gebruikt als sleutel voor SOPS	age

Identiteit en toegang (Microsoft Entra)

Begrip	Uitleg	Lees meer
Microsoft Entra ID	Het identiteitsplatform van Microsoft (voorheen Azure AD): gebruikers, apps, rollen, sign-ins	Entra

Begrip	Uitleg	Lees meer
Service principal	De identiteit van een applicatie in Entra; logt in met een secret, certificaat of federatie	Managed identities
Managed identity	Een door Azure beheerde identiteit voor een resource, zonder credentials die je zelf beheert. *System-assigned* = gekoppeld aan één resource; *user-assigned* = losstaand en herbruikbaar	Managed identities
OIDC (OpenID Connect)	Authenticatieprotocol bovenop OAuth 2.0; een **ID-token** bewijst wie je bent	OpenID
Workload identity federation	Een workload (GitHub Actions, een Kubernetes service account) ruilt een token van zijn eigen uitgever in voor een Entra-token. Geen opgeslagen secret. Vertrouwen via `issuer`, `subject` en `audience` in een *federated credential*	Workload identity federation
Kortlevende vs langlevende credentials	Kortlevend = token dat na minuten tot uren verloopt (veilig bij lekken). Langlevend = secret of certificaat dat maanden geldig is	—
Entra ID Protection	Detecteert risico's op accounts en workload identities, zoals gelekte credentials en verdachte sign-ins	Workload identity risk

Security-scanning

Begrip	Uitleg	Lees meer
Trivy	Open-source scanner voor images, repo's, IaC en clusters: kwetsbaarheden, misconfiguraties, secrets, licenties	Trivy
Trivy Operator	Trivy als operator in het cluster: scant continu en slaat rapporten op als Kubernetes-resources	Trivy Operator

Begrip	Uitleg	Lees meer
CVE	Uniek nummer voor een publiek bekende kwetsbaarheid (bijv. `CVE-2024-1234`)	CVE
Misconfiguratie	Een onveilige instelling, bijv. een container als root of een publiek opslagaccount	—
Secret scanning (GitHub)	GitHub zoekt in repo's naar gelekte tokens en sleutels, en kan een push tegenhouden	GitHub secret scanning
SBOM	Software Bill of Materials: een inventaris van alle onderdelen (packages, versies) in een stuk software	CISA SBOM
SARIF	Standaardformaat (OASIS) voor scanresultaten; GitHub toont SARIF-bestanden als code-scanning-alerts	GitHub SARIF
CIS Benchmark	Set beveiligingsrichtlijnen per product (bijv. Kubernetes) van het Center for Internet Security	CIS Kubernetes

Monitoring en incidentrespons

Begrip	Uitleg	Lees meer
Azure Monitor	Het Azure-platform voor metrics, logs en alerts	Azure Monitor
Log Analytics workspace	De opslag waarin logs terechtkomen en met KQL bevraagd worden; Sentinel draait er bovenop	KQL
KQL	Kusto Query Language: de querytaal voor Log Analytics en Sentinel (pipe-gebaseerd, alleen lezen)	KQL
Diagnostic setting	Instelling op een Azure-resource die bepaalt welke logs waarheen gaan (bijv. Key Vault-AuditEvent naar Log Analytics)	Key Vault logging
AuditEvent	De logcategorie van Key Vault: elke operatie met tijd, resultaat, IP en identiteit	Key Vault logging

Begrip	Uitleg	Lees meer
SIEM	Security Information and Event Management: verzamelt security-logs uit veel bronnen en correleert ze tot incidenten	Sentinel
SOAR	Security Orchestration, Automation and Response: automatisch reageren op incidenten	Sentinel
Microsoft Sentinel	De SIEM + SOAR van Microsoft, op een Log Analytics-workspace; verhuist naar de Defender-portal	Sentinel
Analytics rule	Een detectieregel in Sentinel (meestal een KQL-query op een schema) die alerts en incidenten maakt	Sentinel
Incident	Een gebundeld security-voorval in Sentinel: meerdere alerts en betrokken entiteiten bij elkaar	Sentinel
Playbook / Logic Apps	Geautomatiseerde workflow (trigger + acties, weinig code) die Sentinel start bij een incident, bijv. een secret uitschakelen	Logic Apps
Defender for Key Vault	Onderdeel van Defender for Cloud dat ongebruikelijke toegang tot Key Vaults detecteert	Defender for Key Vault
MITRE ATT&CK	Gratis, wereldwijde catalogus van aanvalstechnieken, gebaseerd op echte aanvallen	MITRE ATT&CK

Certificaten (PKI)

Begrip	Uitleg	Lees meer
Certificaat	Publiek document dat een publieke sleutel koppelt aan een naam (bijv. een domein), ondertekend door een CA	RFC 5280
Private key	De geheime helft van het sleutelpaar; wie die heeft, kan zich voordoen als de eigenaar van het certificaat	—

Begrip	Uitleg	Lees meer
CA (Certificate Authority)	Instantie die certificaten ondertekent	RFC 5280
Certificaatketen	Leaf (je server) → intermediate CA → root CA. Elke laag ondertekent de laag eronder; de keten zelf is publiek	RFC 5280
Revocation (CRL/OCSP)	Een certificaat vóór de verloopdatum ongeldig verklaren; clients controleren dat via een intrekkingslijst (CRL) of online (OCSP)	RFC 5280 · RFC 6960
Certificate Transparency (CT)	Publieke, onveranderbare logs van alle uitgegeven certificaten; monitors waarschuwen je bij certificaten voor jouw domein die je niet zelf aanvroeg	CT
HSM	Hardware Security Module: hardware waarin sleutels worden gemaakt en gebruikt, maar nooit naar buiten kunnen	Key Vault-certificaten
Exporteerbare key	Een private key die uit Key Vault opgehaald mag worden; niet-exporteerbaar = de key verlaat de vault nooit	Key Vault-certificaten

Bronnen

Opgehaald op 15-10-2026. Afwegingen die als eigen afweging zijn gemarkeerd, zijn geen bronfeit.

- CloudNativePG
- CNCF: PostgreSQL operator joins CNCF Sandbox
- External Secrets Operator: Azure Key Vault
- Microsoft Learn: Secrets Store CSI Driver op AKS
- Sealed Secrets
- SOPS
- Trivy
- Microsoft Learn: Microsoft Sentinel overview
- Microsoft Learn: Key Vault logging
- Microsoft Learn: Defender for Key Vault

- Elastic: voorbeeld detectieregel 'Excessive Secret or Key Retrieved'

Aanvullend (Trivy-plekken en certificaten):

- Trivy: integraties (GitHub Actions, Kubernetes operator, VS Code)
- trivy-action (GitHub Action)
- Trivy Operator
- Microsoft Learn: Key Vault-certificaten (key + secret, exporteerbaarheid)
- Microsoft Learn: risico's van workload identities (Leaked Credentials, Suspicious Sign-ins)
- Certificate Transparency: hoe het werkt
- Datadog Security Labs: Kubernetes authentication (geen revocation van certificaten)
- ARMO: client-certificaten niet gebruiken voor gebruikers

Aanvullend (secrets IaC-first):

- Microsoft Learn: workload identity federation (GitHub Actions, Kubernetes; issuer/subject/audience)
- Terraform: sensitive data in state (sensitive, ephemeral, write-only)
- GitHub Docs: secrets in GitHub Actions (en OIDC i.p.v. langlevende credentials)